Licence 3 Informatique - Informatique & Applications
UFR Mathématique et Informatique
Université Paris Cité.

Rapport Projet WayX

Prénom et Nom : **Melissa YAHIATENE**

♦ Recherche du plus court chemin :

Le but de ce TP est de trouver le plus court chemin entre 2 points sur une carte en 2 dimensions, en implémentant deux algorithmes fondamentaux de recherche de chemin optimal : Dijkstra et A*, et de comparer leurs performances.

Le support utilisé pour cette étude est une **carte en deux dimensions**, fournie sous forme de fichier texte. Chaque case de la carte représente un type de terrain associé à un coût de déplacement. L'ensemble de la carte est converti en un **graphe pondéré**, où chaque case devient un sommet, et où les arêtes relient les cases voisines selon une **connexité à 8 voisins**. Le poids d'une arête dépend du coût de déplacement des cases concernées, ce qui permet de modéliser des terrains plus ou moins difficiles à traverser.

L'objectif pédagogique de cette partie est :

1. **Implémenter correctement les deux algorithmes**, en respectant leur logique interne et en tenant compte des spécificités du graphe construit à partir de la carte.
2. **Comparer leurs performances** sur différentes cartes, en analysant l'impact de la structure du terrain, du choix de l'heuristique pour A*, et des coûts associés aux déplacements.

♦ Projet : Graphes et Image (Dijkstra et modélisation) :

Dans ce scénario, l'objectif est d'exploiter la structure implicite d'une image pour construire un graphe pondéré et y appliquer l'algorithme de Dijkstra afin de déterminer le plus court chemin entre deux pixels. L'idée de base repose sur une modélisation naturelle : chaque pixel de l'image est considéré comme un sommet du graphe, et les relations de voisinage entre pixels deviennent des arêtes.
Cette approche permet de transformer une image en une structure exploitable par des algorithmes de graphes. Les arêtes sont pondérées en fonction de la différence d'intensité entre les pixels voisins, ce qui reflète la difficulté ou le coût de passage d'un pixel à un autre. Ainsi, un chemin traversant des zones homogènes sera favorisé, tandis que les zones de forte variation seront pénalisées.

Approche de Résolution :

Pour répondre à ces deux problématiques « ***La recherche du plus court chemin sur une carte textuelle et sur une image entre 2 pixels*** », j'ai choisi de concevoir une application qui combine les deux approches au sein d'une même interface.

En effet, bien que les supports diffèrent (*cartes discrètes / images matricielles*), les deux scénarios reposent sur une même logique algorithmique : la modélisation d'un environnement sous forme de graphe pondéré, et l'utilisation d'algorithmes de recherche de chemin optimal, pour déterminer le plus court chemin entre deux points.

Les algorithmes de Dijkstra et A* sont au cœur de cette démarche. Ils permettent, chacun selon leur stratégie, d'explorer le graphe et de calculer le plus court chemin en tenant compte des coûts associés aux déplacements. Dijkstra garantit une solution optimale sans heuristique, tandis qu'A* accélère la recherche en intégrant une estimation de la distance restante.

En regroupant les deux scénarios dans une seule application, j'ai pu factoriser les structures de données, réutiliser les algorithmes et harmoniser les composants graphiques. Proposant ainsi une solution complète, cohérente et évolutive, avec deux modes distincts mais cohérents :

- **Mode Graphe** : lecture d'un fichier texte, construction d'un graphe à partir de types de terrain, visualisation du chemin.
- **Mode Image** : conversion d'une image en graphe selon les intensités de pixels, sélection interactive des points de départ et d'arrivée, affichage du chemin superposé.

Introduction :

1) Contexte :

Le projet « ***WayX*** » s'inscrit dans le domaine du *pathfinding*, un ensemble de techniques visant à déterminer le chemin optimal entre deux points au sein d'un environnement discret ou continu. Ces méthodes jouent un rôle central dans de nombreux secteurs : navigation robotique, systèmes GPS, jeux vidéo, analyse d'images, modélisation de terrains ou encore intelligence artificielle. Dans tous ces contextes, la capacité à représenter un espace sous forme de graphe et à y calculer efficacement un plus court chemin constitue un enjeu essentiel.

Le projet s'appuie sur cette problématique générale en proposant deux approches complémentaires : la recherche de chemin sur une carte textuelle et la recherche de chemin sur une image convertie en graphe. Ces deux scénarios, bien que différents dans leur support, reposent sur les mêmes fondements théoriques : la modélisation d'un environnement sous forme de graphe pondéré et l'utilisation d'algorithmes classiques tels que Dijkstra ou A*. WayX s'inscrit ainsi dans une démarche d'exploration et de comparaison de ces algorithmes dans des contextes variés, tout en offrant une visualisation claire et interactive des résultats.

2) *Objectif :*

L'objectif du projet est de développer une application capable de :

- Charger une carte ou une image.
- La convertir en graphe pondéré.
- Exécuter des algorithmes de plus court chemin (Dijkstra, A*).
- Visualiser le résultat directement sur l'interface graphique.

3) *Description des Fonctionnalités :*

L'application WayX propose un ensemble de fonctionnalités permettant d'explorer, de visualiser et de comparer des algorithmes de recherche du plus court chemin dans deux environnements distincts : des cartes textuelles et des images en niveaux de gris. Elle se compose de deux modes principaux, accessibles depuis un menu d'accueil simple et intuitif.

1. Menu principal :

Le menu d'accueil permet à l'utilisateur de choisir entre deux modes d'utilisation :

- **Mode Graphe :** recherche de chemin sur une carte textuelle.
- **Mode Image :** recherche de chemin sur une image convertie en graphe.

Ce menu constitue le point d'entrée de l'application et assure une navigation fluide entre les différents modules.

2. Mode Graphe :

Ce mode permet de charger un fichier texte décrivant une carte et de visualiser le graphe correspondant.

Fonctionnalités principales :

- Chargement d'une carte au format texte (types de terrain, couleurs, coûts, départ et arrivée).
- Construction automatique du graphe à partir de la carte (connexité 8 voisins, pondération selon les terrains).
- Affichage graphique de la carte, avec coloration des différents types de cases.
- Exécution de Dijkstra pour calculer le plus court chemin.
- Exécution de A* avec heuristique pour accélérer la recherche.
- Visualisation du chemin trouvé, superposé sur la carte.
- Affichage optionnel des explorations, permettant de voir les nœuds visités par l'algorithme.
- Bouton Reset pour réinitialiser l'affichage et relancer une recherche.
- Retour au menu principal à tout moment.

Ce mode met en avant la logique de pathfinding sur des terrains hétérogènes.

3. Mode Image :

Ce mode permet de transformer une image en niveaux de gris en un graphe pondéré, puis d'y appliquer les algorithmes de plus court chemin.

Fonctionnalités principales :

- Chargement d'une image (PNG, JPG, etc.).
- Conversion automatique en matrice d'intensités (0–255).
- Construction du graphe en 4-connexité, où chaque pixel devient un sommet.
- Pondération des arêtes selon la différence d'intensité entre pixels voisins.
- Affichage de l'image redimensionnée dans la fenêtre.
- Sélection interactive du départ et de l'arrivée par simple clic sur l'image.
- Exécution de Dijkstra pour trouver le chemin optimal.
- Exécution de A* avec heuristique euclidienne.
- Affichage du chemin directement sur l'image.
- Bouton Reset pour effacer le chemin et recommencer.
- Retour au menu principal.
- Ce mode illustre l'application des algorithmes de graphes au traitement d'images.

4. Algorithmes intégrés :

L'application implémente deux algorithmes de référence :

Dijkstra :

- Trouve le plus court chemin sans heuristique.
- Garantit une solution optimale.
- Explore largement le graphe.

A* :

- Version optimisée de Dijkstra.
- Utilise une heuristique (Euclidienne/Manhattan/Zéro dans WayX).
- Réduit considérablement le nombre de nœuds explorés.
- Plus rapide dans la majorité des cas.

Les deux algorithmes sont entièrement visualisables dans l'interface.

4) Réalisations Techniques :

4.1. Choix technologiques :

- Langage : Java.
- Interface graphique : Swing.
- Structure du projet : MVP (Model / View / Presenter).
- Algorithmes : Dijkstra, A* avec heuristiques.

4.2. Architecture générale :

L'architecture de mon application repose sur le modèle **Model-Vue-Presenter (MVP)**.
Ce pattern permet de séparer les responsabilités entre la logique métier, l'interface graphique et la coordination des actions, facilitant ainsi la maintenance, l'évolution et les tests du projet.

Les interactions entre les composants s'organisent comme suit :

- ***Le composant Modèle (Model) :***
  Contient toute la logique métier et les structures de données :

  - **algorithms :**
    - Dijkstra.java
    - AStar.java

    Implémentation des algorithmes de plus court chemin.

  - **heuristics :**
    - Heuristic.java (interface)
    - EuclideanHeuristic.java
    - ManhattanHeuristic.java
    - ZeroHeuristic.java

    Définition des heuristiques utilisées par A*.

  - **util :**
    - Graph.java, Vertex.java, Edge.java
    - MapManager.java : chargement des cartes textuelles
    - PathExplorer.java : exploration des chemins

    Structures de graphe et outils de manipulation.

- ***Le composant Vue (View) :***
  Gère l'interface graphique :

  - **GraphView.java** : vue pour les cartes textuelles
  - **ImageView.java** : vue pour les images
  - **Board.java** : panneau d'affichage des cartes
  - **ImageBoard.java** : panneau d'affichage des images

  Chaque vue est responsable de l'affichage et de la gestion des événements utilisateur.

- ***Le composant Présentateur (Presenter) :***
  Coordonne les interactions entre la vue et le modèle :

  - **PathPresenter.java :**
    - Reçoit les actions de la vue
    - Exécute les algorithmes
    - Met à jour l'affichage du chemin

✦ En plus d'un module pour le traitement d'images :

- ***ImageHandler :***
  Spécifique au mode image :

  - **ImageLoader.java** : conversion d'image en matrice d'intensités.
  - **GraphBuilder.java** : transformation de la matrice en graphe.
  - **ImageBoard.java** : affichage graphique de l'image et du chemin.

- Et enfin **WayXApp.java** : point d'entrée de l'application, menu principal

5) *Analyse des Algorithmes implémentés :*

5.1. Présentation générale des algorithmes :

Dans le cadre de ce projet, deux algorithmes de recherche du plus court chemin ont été implémentés : **Dijkstra** et **A*.** Ces deux méthodes reposent sur la même idée : explorer un graphe pondéré afin de déterminer le chemin le plus court entre un sommet de départ et un sommet d'arrivée.

La différence principale réside dans la manière dont ils choisissent le prochain nœud à explorer.

5.2. Algorithme de Dijkstra :

L'algorithme de Dijkstra est une méthode gloutonne qui calcule la distance minimale entre un point de départ et tous les autres sommets du graphe.

- **Fonctionnement :**
  - À chaque étape, l'algorithme sélectionne le sommet non visité ayant la plus petite distance temporaire.
  - Il met à jour les distances de ses voisins si un chemin plus court est trouvé.
  - Le processus continue jusqu'à atteindre le sommet d'arrivée.

- **Propriétés :**
  - Optimal : garantit le plus court chemin.
  - Indépendant de la structure du graphe : aucune heuristique nécessaire.
  - Exploration large : visite souvent des zones inutiles.

5.3. Algorithme A* :

A* est une extension de Dijkstra qui introduit une heuristique pour guider la recherche vers l'objectif.

Il sélectionne le prochain sommet selon :

$$f(n)=g(n)+h(n)$$

Où :
***g(n)* = coût réel depuis le départ,**
***h(n)* = estimation du coût restant (heuristique).**

- **Propriétés :**
  - Plus rapide que Dijkstra lorsque l'heuristique est pertinente.
  - Optimal si l'heuristique est admissible.
  - Exploration ciblée, concentrée vers l'objectif.

5.4. Heuristiques implémentées :

✦ ***Heuristique euclidienne :***

$$h(n) = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

- Adaptée aux environnements 2D continus ou aux images.
- Admissible : garantit l'optimalité de A*.
- Très efficace dans WayX.

✦ ***Heuristique de Manhattan :***

$$h(n) = | x2 - x1 | + | y2 - y1 |$$

- Pertinente en 4-connexité.
- Moins précise en 8-connexité.
- Reste plus rapide que Dijkstra.

✦ ***Heuristique nulle (Zéro) :***

$$h(n) = O$$

- ***A* ≡ Dijkstra***
- Utilisée comme référence.

5.5. Comparaison des performances sur différentes cartes :

Pour évaluer les algorithmes, plusieurs cartes textuelles ont été testées, variant par :

- Taille,
- Densité d'obstacles,
- Diversité des terrains,
- Distance entre départ et arrivée.

♦ **Résultats pour Dijkstra :**

- Explore un grand nombre de nœuds.
- Temps d'exécution élevé sur les grandes cartes.
- Chemin optimal garanti.
- Insensible à la position de l'objectif.

- **Résultats pour A* (heuristique euclidienne) :**

  - Nombre de nœuds explorés fortement réduit.
  - Temps d'exécution nettement inférieur.
  - Chemin optimal conservé.
  - Recherche dirigée vers l'objectif.

- **Résultats pour A* (Manhattan) :**

  - Performances intermédiaires.
  - Moins efficace que l'euclidienne en 8-connexité.
  - Toujours plus rapide que Dijkstra.

5.7. Réflexion sur les cas réels d'utilisation :

Les observations faites dans WayX se retrouvent dans des applications concrètes :

- **Quand privilégier Dijkstra ?**

  - Lorsque aucune heuristique fiable n'existe.
  - Lorsque l'on souhaite connaître toutes les distances depuis une source.
  - Dans des graphes abstraits (réseaux sociaux, réseaux électriques).
  - Dans des environnements où la position de l'objectif n'a pas de sens géométrique.

- **Quand privilégier A* ?**

  - Lorsque la position de l'objectif est connue.
  - Lorsque l'on dispose d'une heuristique cohérente.
  - Dans les systèmes nécessitant une réponse rapide :
    - GPS et navigation routière,
    - Robotique mobile,
    - Jeux vidéo (IA, déplacements d'unités),
    - Traitement d'images (chemins de moindre variation),
    - Planification de trajectoires.

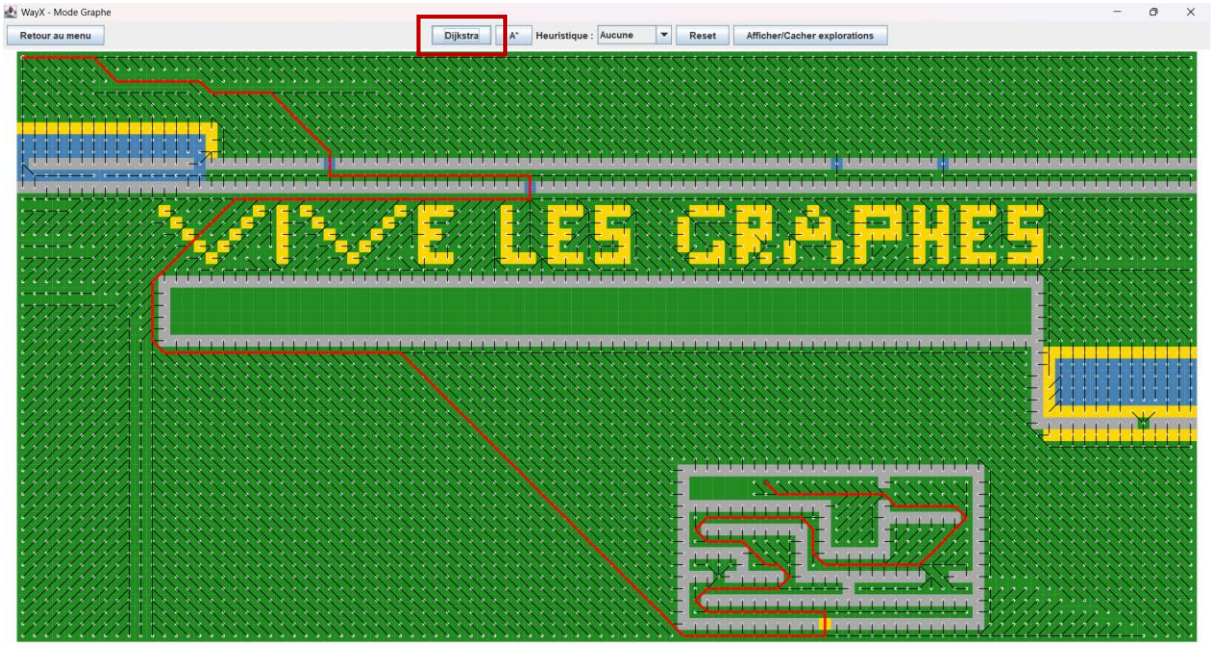
  A* est aujourd'hui l'algorithme le plus utilisé dans les applications interactives et temps réel.

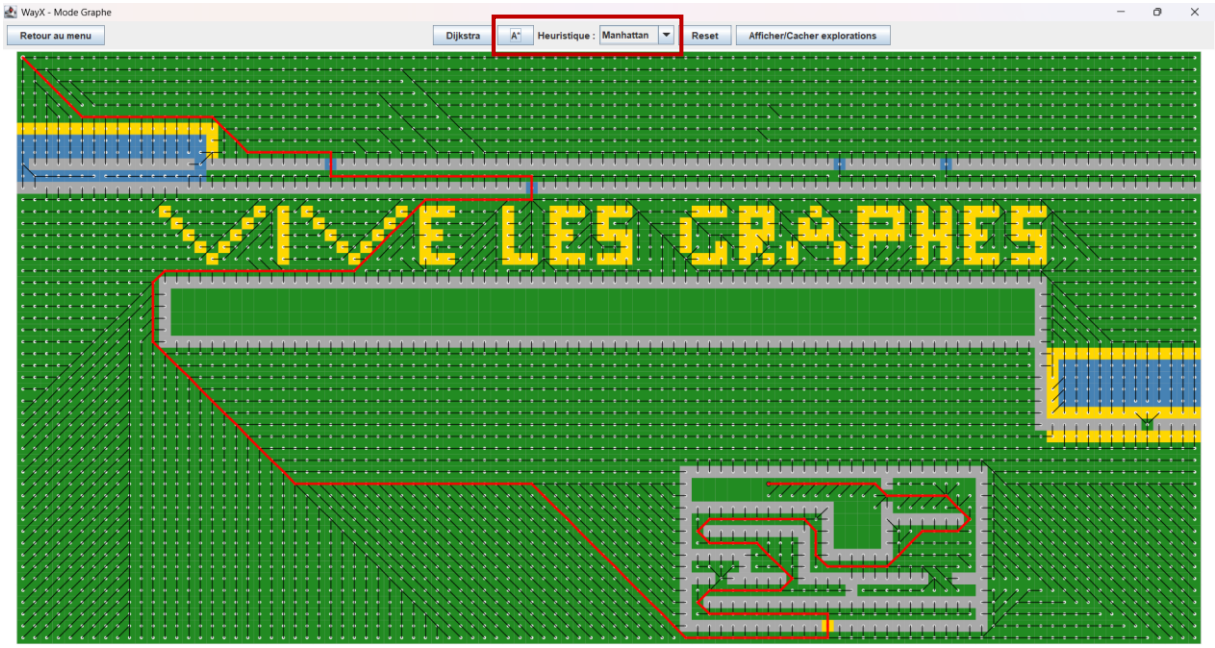
7.8. Conclusion de l'analyse :

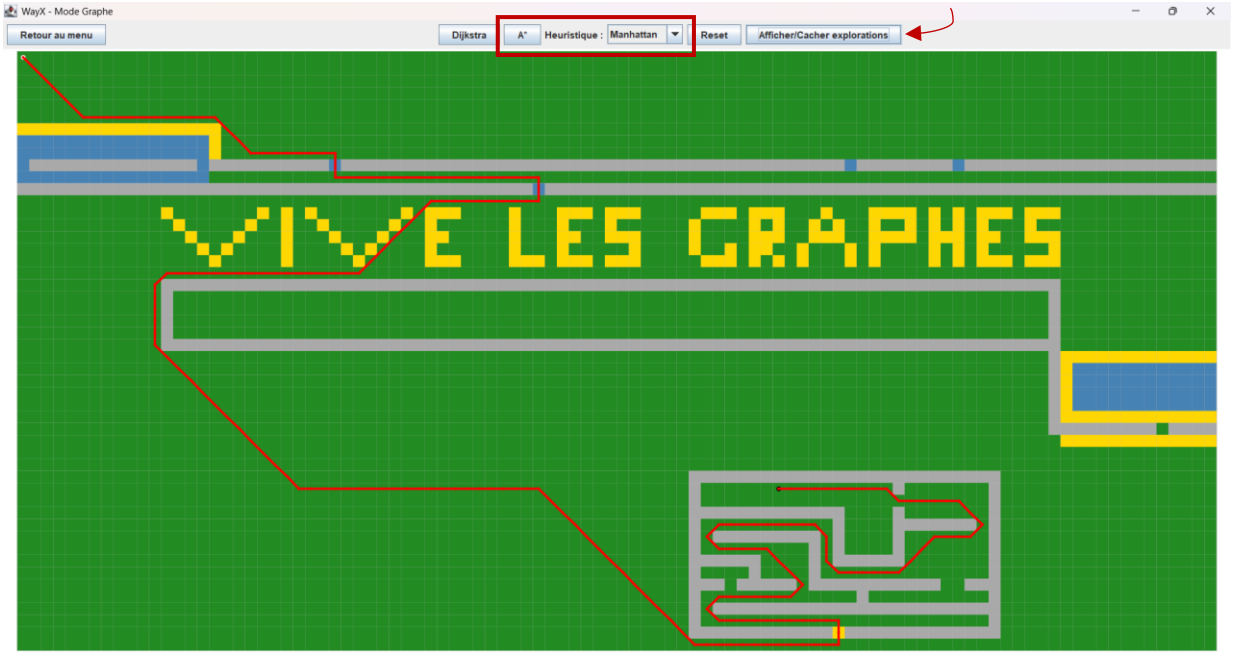
L'étude menée dans WayX montre clairement que :

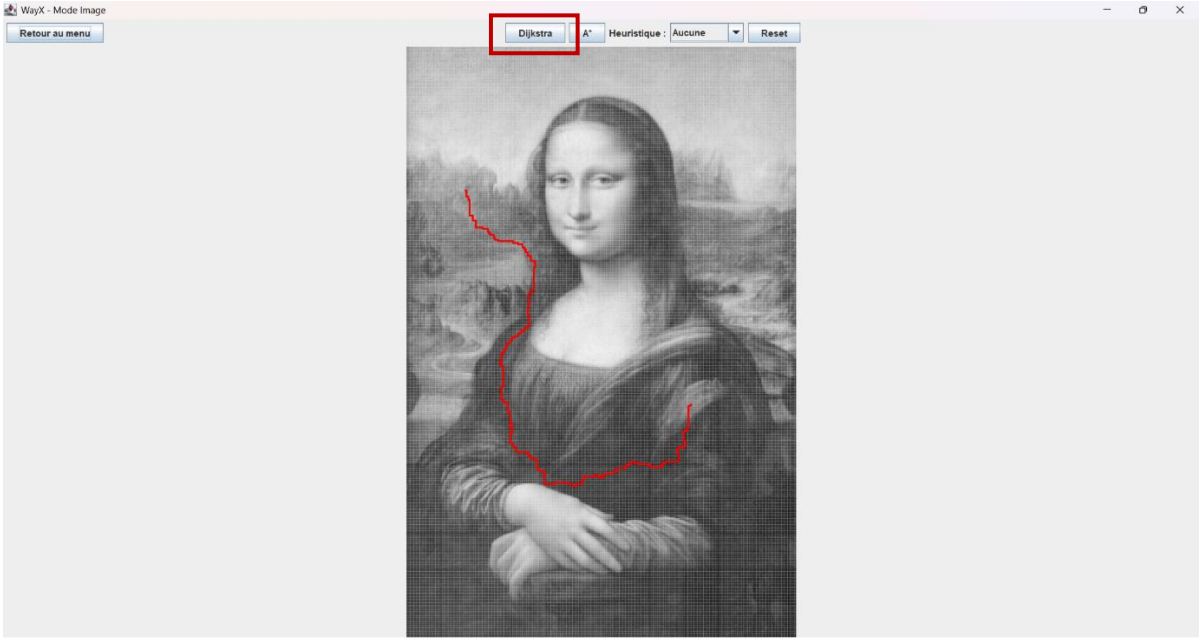
- Dijkstra est robuste et fiable, mais explore trop largement.
- A*, avec une heuristique adaptée, est nettement plus performant tout en conservant l'optimalité.
- Le choix de l'heuristique influence fortement la rapidité, mais pas la qualité du chemin si elle reste admissible.
- Cette analyse met en évidence l'importance du choix algorithmique et de la modélisation du graphe dans toute application de pathfinding.

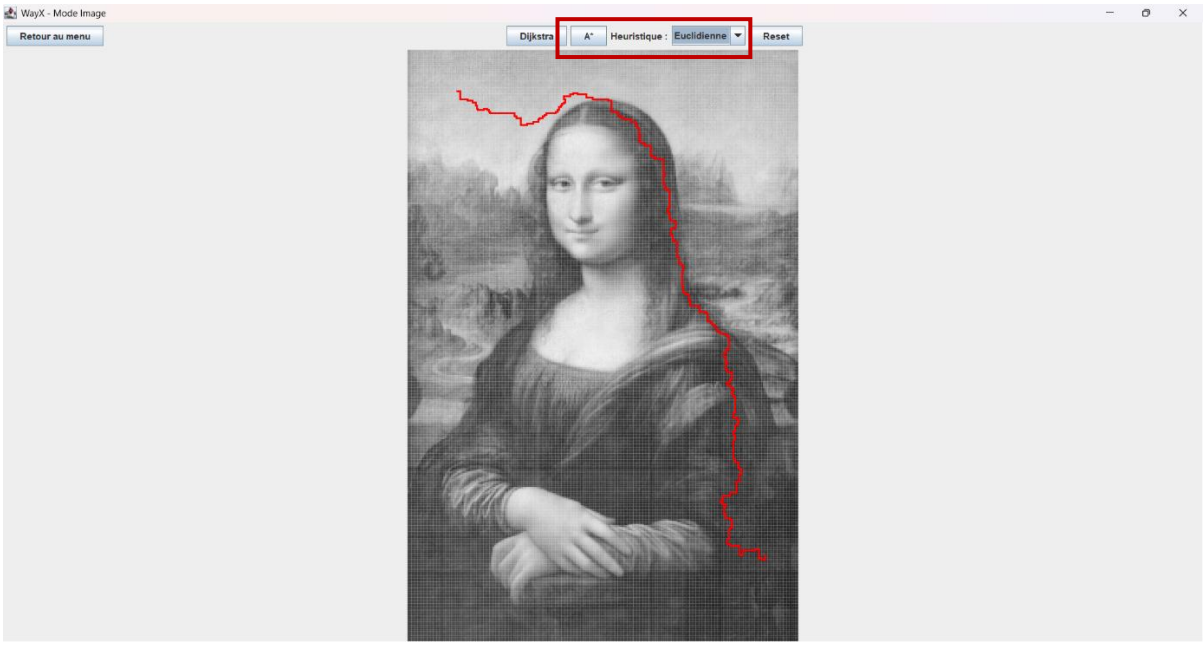
❖ ***Exemple de test sur le graphe et l'image fournis (graph.txt / Mona_Lisa.png) :***






WayX - Mode Graphe
Retour au menu
Dijkstra
A*
Heuristique :
Manhattan
Reset
Afficher/Cacher explorations
VIVE LES GRAPHES


WayX - Mode Image
Retour au menu
Dijkstra
A*
Heuristique :
Aucune
Reset


WayX - Mode Image
Retour au menu
Dijkstra
A*
Heuristique :
Euclidienne
Reset